Management System ISO 9001:2015
Level III Recognition for Mastery in Quality Management 2022
THE Impact Rankings 2023 +1001
Dark Green School Certified
World University Rankings Ranked 684
The WORLD UNIVERSITY RANKINGS for INNOVATION

ACTIVITY 21 – HASHING IN JAVA USING HASHMAP

Activity 1: The Simple Fruit Basket (Basic CRUD Operations)

Objectives

- **Apply** basic HashMap methods (put(), get(), and containsKey()) to store and retrieve data.
- **Demonstrate** how to add new items and update values under an existing key.

Instructions

You are tasked with building a digital fruit basket application that tracks how many fruits of each type you have.

1. **File Submission:** You must submit your raw .java file to the online school portal. Programs that do not compile will automatically receive a zero.
2. **Naming Convention:** Name your file and class following this exact structure: FruitBasket_[YourFirstName]_[YourLastName].java. For example: FruitBasket_Juan_DelaCruz.java.
3. **Required Code Tasks:**
    - Create a HashMap<String, Integer> where the **Key** is the fruit name (String) and the **Value** is the quantity (Integer).
    - Add three initial fruits: "Apples" with 10, "Bananas" with 5, and "Oranges" with 8.
    - Print the entire fruit basket.
    - Update the number of "Apples" to 15 because you bought more.
    - Use containsKey() to check if "Grapes" exist in your basket. Print an appropriate message based on the result.
4. **Screenshot Requirements:** Take a single screenshot of your console output showing the original basket, the updated apples, and the search check for grapes.

Program Output

```
--- My Fruit Basket ---
Initial Basket: {Apples=10, Bananas=5, Oranges=8}

Updating Apples:
New Apple Count: 15

Checking for Grapes...
Does the basket have grapes? false

Final Basket: {Apples=15, Bananas=5, Oranges=8}
Press any key to continue . . .
```

Activity 2: Vocabulary Lookup (Reading & Checking Data)

Objectives

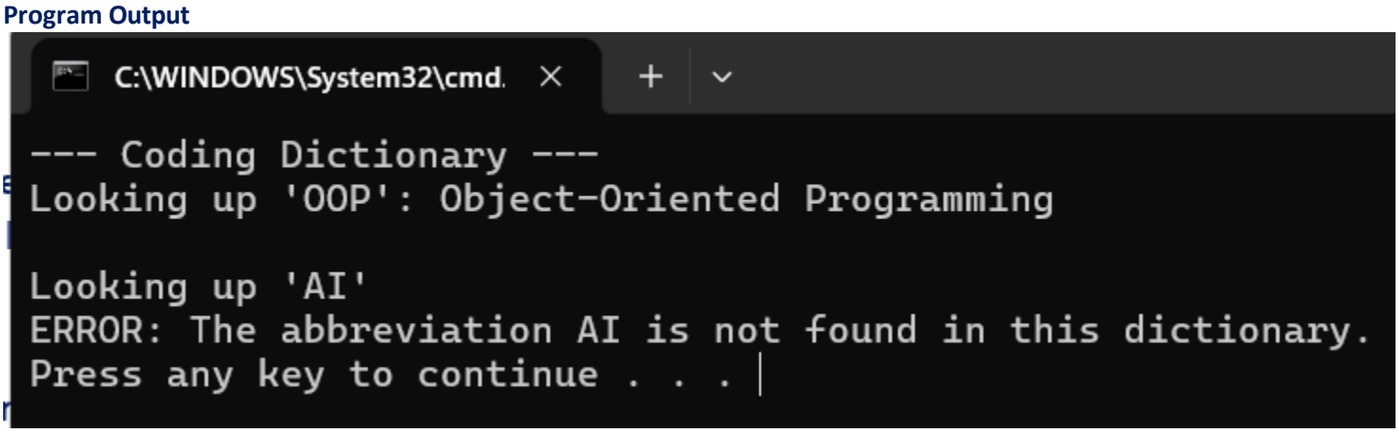
- **Utilize** a HashMap as a lookup dictionary to fetch values based on string inputs.
- **Handle** missing keys safely by validating user inputs before retrieval.

Instructions

Create a mini computer programming dictionary that translates common coding acronyms into their full definitions.

1. **File Submission:** Submit the final standalone .java file to the online portal.
2. **Naming Convention:** Name your file and class following this exact structure: CodingDictionary_[YourFirstName]_[YourLastName].java. For example: CodingDictionary_Juan_DelaCruz.java.
3. **Required Code Tasks:**
    - Create a HashMap<String, String> where both the Key and Value are Strings.
    - Add three pairs:
        - "OOP" ➔ "Object-Oriented Programming"
        - "JDK" ➔ "Java Development Kit"
        - "RAM" ➔ "Random Access Memory"
    - Fetch and print the definition of "OOP" using the .get() method.
    - Try to fetch a definition for an abbreviation that doesn't exist, like "AI". Show how Java returns null or a friendly "Not Found" error message.
4. **Screenshot Requirements:** Capture your command line showing the printed definition lookup and the handling of the missing word.

Program Output



Management System ISO 9001:2015
Level III Recognition for Mastery in Quality Management 2022
THE Impact Rankings 2023 +1001
Dark Green School Certified
World University Rankings Ranked 684
The WORLD UNIVERSITY RANKINGS for INNOVATION

Activity 3: Contact List Clean-up (Deleting Data)

Objectives

- **Implement** the .remove() method to erase pairs from memory using keys.
- **Track** the size changes of a collection using the .size() method.

Instructions

Manage a small list of phone contacts. Your job is to initialize the list, check its size, remove a contact who changed their number, and verify the changes.

1. **File Submission:** Submit your primary execution driver class as a raw .java file.
2. **Naming Convention:** Name your file and class following this exact structure: ContactList_[YourFirstName]_[YourLastName].java. For example: ContactList_Juan_DelaCruz.java.
3. **Required Code Tasks:**
    - Create a HashMap<String, String> where the Key is a person's name (String) and the Value is their phone number (String).
    - Add three contacts: "Mom" ("555-0123"), "Dad" ("555-4567"), and "Alex" ("555-9876").
    - Print out the current total size of the map using .size().
    - Remove "Alex" from the contact list using the .remove() method.
    - Print the updated contact list and its final size to prove it was successfully erased.
4. **Screenshot Requirements:** Provide a screenshot displaying your code output showing the list size before and after dropping the contact.

Program Output

```
---Contact List Example---
Total contacts stored: 3

Removing contact 'Alex'...
updated List: {Dad=555-4567, Mom=555-0123}
Total contacts stored now: 2
Press any key to continue . . .
```













Reflective Questions

1. When you printed out your entire HashMap at once, did the items print out in the exact order you added them? Why do you think a HashMap behaves this way compared to a standard ArrayList?
   When I printed the HashMap, it was sorted and printed in alphabetical order. Compared to ArrayList, HashMap is more optimized with keys and value pair.

2. What happens if you try to call .put("Apples", 20) on a map that already has an entry for "Apples"? Does it create a second entry, or does it do something else?
   When putting another put(“Apples”, 20), it was not making another entry for the Apples. Instead, it overwrites the existing value.

3. Imagine you have a lookup list with 5,000 items. Why is using a HashMap with a unique key faster than using a simple for loop to scan an array list item by item?
   HashMap Is optimized to do lookups even with a lot of entries as HashMap searches for the key instead of the value.

Grading Rubric (100 Points Total)

Criteria	Exceptional (20-17 pts)	Proficient (16-13 pts)	Developing (12-9 pts)	Inadequate (8-0 pts)
Functional Correctness	Program compiles with no warnings and runs all the steps correctly.	Program compiles and runs, but misses one minor logical instruction step.	Program compiles but crashes or outputs incorrect logic sequences during run.	Code fails to compile completely or misses the entire point of the task.
HashMap Logic Application	Correctly maps keys/values and perfectly utilizes target methods (put, get, remove, etc.).	Correctly uses maps but uses overly complicated code where built-in features exist.	Mixes up keys and values or struggles with simple data type syntax rules.	Fails to use a HashMap or uses incorrect collections frameworks entirely.
Naming Conventions & Formats	Followed the exact class/file naming schema; class name includes student name.	Followed naming schemes but had minor spelling typos in the class name layout.	Misnamed components significantly or forgot to embed personal name labels.	Completely ignored file naming guidelines.
Output Screenshots	Crisp, clear console window captures demonstrating successful runs of the goals.	Screenshots provided but cropped poorly or missing one intermediate print test.	Missing key screenshots or images are too blurry to read your output values.	No screenshots or output proof files were included with the submission.
Reflective Questions	Answers show great baseline understanding of key, value, and hashing behavior elements.	Answers are correct but too brief, leaning on text answers without explanations.	Reflections indicate minor confusion about how maps internally track data items.	Answers are completely missing, copied, or wrong.

100 TOTAL PTS